Analyzing Loan Default Risk:
A Statistical and Probabilistic Perspective

Pragnya Upendra Pathak
A69027339
prpathak@ucsd.edu

Vivekanand Gyanchand Sahu
A69027331
vsahu@ucsd.edu

***Abstract*: The evaluation of loan default risk remains a crucial component for investors, lenders, and financial institutions as the financial environment changes. To enhance prediction accuracy and lending process decision-making, this project models and analyzes the risk of loan default using a thorough statistical and probabilistic framework. The main goals of this study are to assess the probability of loan default and examine statistical trends that may influence loan outcomes. The objective is to apply probability and statistical concepts to enhance lending processes and risk assessment approaches.**
***Keywords:* Baye's Decision Rule, Binary Classification, Maximum Likelihood Estimation, Data Pre-processing, Data Analysis, Feature Engineering, KNN (K Nearest Neighbors), Isolation Forest Anomaly Detection.**

I. BACKGROUND AND INTRODUCTION

The modern banking sector, equipped with advanced technology and data management systems, meticulously accumulates and retains an extensive array of information about each individual client. This repository of data is considered invaluable and is seldom discarded, representing a treasure trove of insights for various financial analyses. One of the domains where this amassed data holds significant promise is in fortifying the credit risk assessment process.

In the realm of financial services, lending institutions grapple with the perpetual challenge of evaluating and mitigating risks associated with their loan portfolios. The meticulous examination of loan default risk, a scenario that unfolds when borrowers fail to make timely payments, is a pivotal aspect of the overarching risk management strategy. Leveraging sophisticated data analytics techniques on this wealth of information is poised to revolutionize the accurate estimation of the likelihood of loan defaults, offering a proactive approach to risk mitigation within the industry.

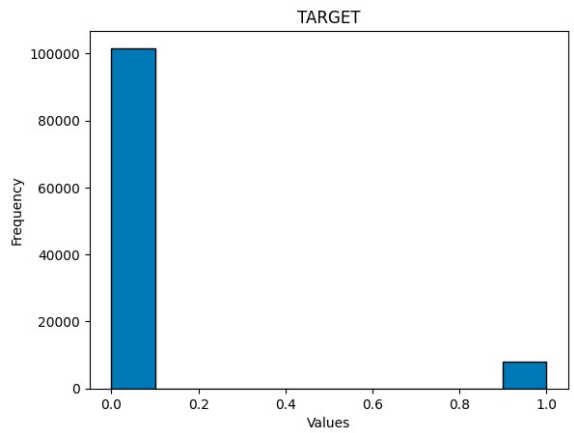
The project analyzes the financial data of individuals and classifies them into categories based on the probability of loan default risk. It delves into various methodologies, models, and tools rooted in statistical and probabilistic principles to provide lenders with a robust framework for evaluating and predicting the likelihood of loan default. By leveraging quantitative techniques, this approach aims to enhance the accuracy of risk assessments, thereby enabling more informed and data-driven lending decisions.

II. DATASET

For this project, we used a subset of the 'Home Credit Default Risk' dataset available on Kaggle. This dataset consisted of 307511 rows corresponding to different loan IDs and 121 columns corresponding to different features. We used the static data for all loan applications where each row represents an application and is uniquely identified by the 'SK_ID_CURR' column. The 'TARGET' column corresponds to the output variable of the classification problem. Each entry is assigned '0' if the loan has been repaid and '1' if the loan has been defaulted. The dataset consists of multiple categorical and numerical features. An example of categorical features is CODE_GENDER: this is the gender of the client and has three categories: 'M', 'F', 'XNA'. One of the numerical features in the dataset is 'AMT_CREDIT', which is the credit amount of the loan. This feature has values ranging from 45000.0 to 4050000.0. We delve further into each of the columns and employ data manipulation techniques on them to make them suitable inputs that can be fed to our model.

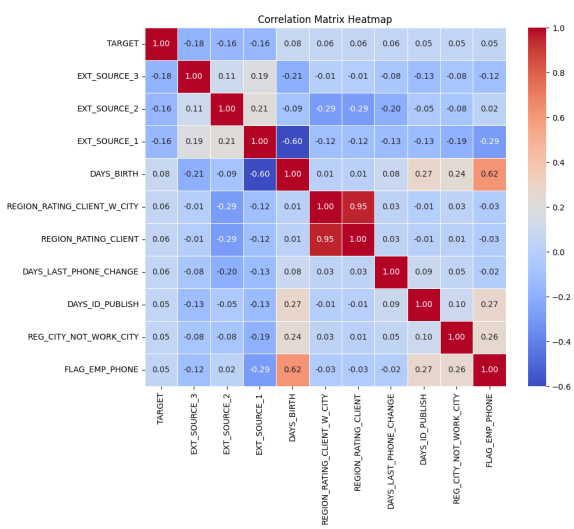
III. DATA ANALYSIS

The data was analyzed after inspecting all the columns and their data types. The entries that had null values were deleted. Upon inspecting our target column, it was observed that the classes were imbalanced. This highlighted that very few people fall under the loan default category (class 1). To calculate the prior probabilities of classes, a histogram plot is made.

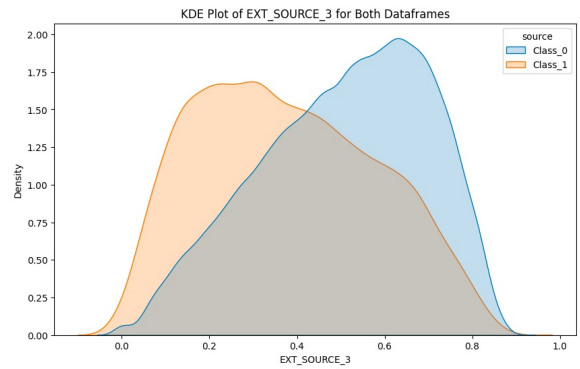


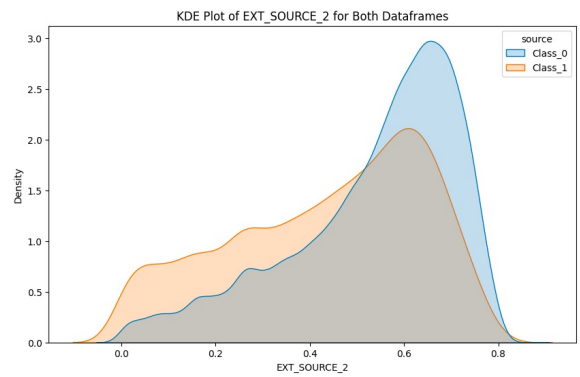
The prior of class 0 = (number of samples in class 0) / (Total number of samples) The prior of class 1 = (number of samples in class 1) / (Total number of samples)

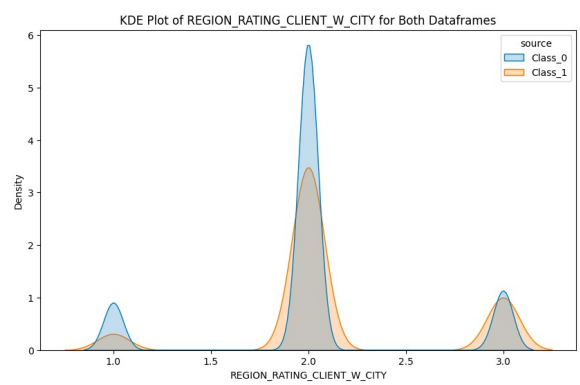
The dataset has many features, out of which only some features have information that significantly differentiates the classes. To extract these features, we evaluated the correlation matrix and arranged the features in descending order of their absolute values. Upon comparing the performance vs computation complexity of the features, we finalized the 10 most informative features for building the model.



To understand how a particular feature performs for two different classes, we divide the dataset into two parts with respect to the classes in the 'TARGET' column. The Kernel Density Estimation (KDE) plots are used to estimate the probability density function of a continuous random variable. The lesser the overlapping area of the two plots, the better is the feature to distinguish the classes.







IV. PREDICTIVE MODELING

This problem can be treated as a binary classification problem with output classes '0' and '1'. To solve the classification task, we started with a basic Maximum Likelihood model and combined it with Baye's Decision Rule.

Maximum Likelihood & Baye's Decision Rule

Maximum Likelihood Estimation (MLE) and Baye's Decision Rule (BDR) are statistical concepts used together in classification problems. MLE estimates

model parameters, while BDR is a decision-making framework based on probabilities and costs. MLE is used to estimate probability distribution parameters for classification, while BDR considers the probability of an observation belonging to a specific class and the costs of misclassification.

For Maximum Likelihood Estimation, since we have more than 30 samples for each class, using the Central Limit Theorem, we approximated the classes to their equivalent Gaussian distributions. For a Gaussian distribution, the parameters are mean and standard deviation. The maximum likelihood estimates of these are the sample mean and standard deviation. Using these estimates, we fit the data to Gaussian. The estimators for mean and covariance are as follows:

$$\hat{\boldsymbol{\mu}} = \frac{1}{n}\sum_{k=1}^{n}\mathbf{x}_k \qquad \widehat{\boldsymbol{\Sigma}} = \frac{1}{n}\sum_{k=1}^{n}(\mathbf{x}_k - \hat{\boldsymbol{\mu}})(\mathbf{x}_k - \hat{\boldsymbol{\mu}})^t.$$

Using these estimators, we fit a multivariate Gaussian to obtain the class conditional probability.

$$p(x;\mu,\Sigma) = \frac{1}{(2\pi)^{n/2}|\Sigma|^{1/2}}\exp\left(-\frac{1}{2}(x-\mu)^T\Sigma^{-1}(x-\mu)\right).$$

Once we get the class conditional probability, we fit each test point to the class conditional Gaussian and then calculate the posterior probability of each class by using the class conditional probability and the prior probability.

$$P(\omega_i|\mathbf{x},\mathcal{D}) = \frac{p(\mathbf{x}|\omega_i,\mathcal{D})P(\omega_i|\mathcal{D})}{\sum_{j=1}^{c} p(\mathbf{x}|\omega_j,\mathcal{D})P(\omega_j|\mathcal{D})}$$

Finally, we use Baye's Decision Rule to determine which class has the highest posterior probability. The class with the higher posterior probability is assigned to that particular data point.

K Nearest Neighbors

The second classification technique we used was the KNearestNeighbors algorithm. KNearestNeighbors is a non-parametric classification algorithm. In the training phase, it first looks at the complete dataset and interprets it as an n-dimensional vector. Once it gets a test data point, it calculates the Euclidean distance between the test point and the train data points. After getting the k nearest neighbors of the test point, it checks for the most prevalent class in the neighborhood of the point and assigns it to the data point.

Isolation Forest Anomaly Detection

Our problem statement is to determine the probability of a person to default on a loan. It is common knowledge that the chance that an applicant will default on their loan is very low. This can also be seen from the class distribution plotted in the previous section, where the number of rows having 'TARGET' value as '1' is almost 10% of the rows having 'TARGET' value as '0'. Hence, it made sense to see it as an anomaly detection problem. For this reason, we applied the Isolation Forest Anomaly Detection algorithm. In this algorithm, the majority class is considered normal, and the minority class is termed as an anomaly. In the scope of this project, the TARGET=1 class is considered an anomaly.

V. RESULTS AND CONCLUSION

We trained three different models in this project. For maximum likelihood estimation with Baye's Decision Rule we considered two cases; the first case was building the model with all 121 features. The second case was to build a model using only the 10 most important features based on their correlation with the target variable.

Model	MLE & BDR using 121 features	MLE & BDR using 10 best features	K Nearest Neighbors	Isolation Forest Anomaly Detection
Accuracy	0.52	0.64	0.76	0.89

From the above table we can see that the accuracy increases if we use only the most informative features. We can conclude that using less correlated features can negatively impact the model

performance as it contributes non distinguishing information which leads to decrease in model accuracy.

The MLE & BDR model is a very naive model that takes into consideration only one best model corresponding to the best estimate of the parameters. Due to this there is a huge loss of information from the other estimates which might contain some information. Hence, the accuracy of this model is the lowest. KNN is a more sophisticated way to classify the data which leads to higher accuracy.

The accuracy of the Isolation Forest Anomaly Detection algorithm is the highest. This confirms our belief that this problem can be best worked as an anomaly detection problem compared to a binary classification problem since the data was skewed towards class ‘0’.

VI. REFERENCES

[1] R. O. Duda, P. E. Hart, and D. G. Stork, "Pattern Classification," John Wiley & Sons, 2001.

[2] A. Łuczak, M. Ganzha, and M. Paprzycki, "Probability of Loan Default—Applying Data Analytics to Financial Credit Risk Prediction," 2021. [Online]. Available: 10.1007/978-981-16-0730-1_1.

[3] J. Brownlee, “Imbalanced Classification with Python. Machine Learning Mastery”, 2020.

CODE:

```
import zipfile
import pandas as pd
import os
import numpy as np
from imblearn.over_sampling import SMOTE
from sklearn.model_selection import train_test_split
from sklearn.ensemble import
RandomForestClassifier
from sklearn.metrics import classification_report
from imblearn.under_sampling import
RandomUnderSampler

def unzip_csv(zip_file_path,csv_file_name):
  with zipfile.ZipFile(zip_file_path, 'r') as zip_ref:
      zip_ref.extract(csv_file_name, path='.')

zip_file_path = '/content/application_data.csv.zip'
csv_file_name = 'application_data.csv'
unzip_csv(zip_file_path,csv_file_name)
application_data_df = pd.read_csv(csv_file_name)
os.remove(csv_file_name)

application_data_df.shape

application_data_df['FLAG_OWN_REALTY'].value
_counts()

application_data_df['CODE_GENDER'].value_count
s()

application_data_df['AMT_CREDIT'].max()

application_data_df['AMT_CREDIT'].min()

correlation_matrix = application_data_df.corr()

correlation_with_target =
correlation_matrix['TARGET'].abs()

sorted_correlation =
correlation_with_target.sort_values(ascending=False)
d = dict(sorted_correlation)
sorted_keys = sorted(d, key=d.get)
sorted_keys = sorted_keys[::-1]
sorted_keys = (sorted_keys[0:11])
print(sorted_keys)

application_data_df =
application_data_df[sorted_keys]

correlation_matrix = application_data_df.corr()

plt.figure(figsize=(10, 8))
heatmap = sns.heatmap(correlation_matrix,
annot=True, cmap='coolwarm', fmt=".2f",
linewidths=.5)

heatmap.set_xticklabels(heatmap.get_xticklabels(),
fontsize=12)
heatmap.set_yticklabels(heatmap.get_yticklabels(),
fontsize=12)

plt.title('Correlation Matrix Heatmap', fontsize=16)
plt.show()

from sklearn.preprocessing import LabelEncoder
import numpy as np

label_encoder = LabelEncoder()

numerical_columns =
application_data_df.select_dtypes(include=['int64',
'float64']).columns
categorical_columns = [col for col in
application_data_df.columns if col not in
numerical_columns]
for columns in categorical_columns:
  application_data_df[columns] =
label_encoder.fit_transform(application_data_df[colu
mns])

application_data_df =
application_data_df.replace([np.inf, -np.inf], np.nan)

application_data_df = application_data_df.dropna()

X_train, X_test, y_train, y_test =
train_test_split(application_data_df[[col for col in
application_data_df.columns if col!='TARGET']],
application_data_df['TARGET'], test_size=0.2,
random_state=42)
smote = SMOTE(random_state=42)
X_train, y_train = smote.fit_resample(X_train,
y_train)

print("X_train shape:", X_train.shape)
```

```
print("X_test shape:", X_test.shape)
print("y_train shape:", y_train.shape)
print("y_test shape:", y_test.shape)

import matplotlib.pyplot as plt
import pandas as pd
import seaborn as sns

application_data_df_0['source'] = 'Class_0'
application_data_df_1['source'] = 'Class_1'
combined_df = pd.concat([application_data_df_0,
application_data_df_1])

plt.figure(figsize=(10, 6))
sns.kdeplot(data=combined_df,
x='DAYS_LAST_PHONE_CHANGE', hue='source',
common_norm=False, fill=True)
plt.title('KDE Plot of
DAYS_LAST_PHONE_CHANGE for Both
Dataframes')
plt.xlabel('DAYS_LAST_PHONE_CHANGE')
plt.ylabel('Density')
plt.show()

import numpy as np
from scipy.stats import multivariate_normal

class_1_indices = (y_train == 0)
class_2_indices = (y_train == 1)
class_1_data = X_train[class_1_indices]
class_2_data = X_train[class_2_indices]

mean_class_1 = np.mean(class_1_data, axis=0)
cov_class_1 = np.cov(class_1_data, rowvar=False)

mean_class_2 = np.mean(class_2_data, axis=0)
cov_class_2 = np.cov(class_2_data, rowvar=False)

dist_class_1 =
multivariate_normal(mean=mean_class_1,
cov=cov_class_1, allow_singular=True)
dist_class_2 =
multivariate_normal(mean=mean_class_2,
cov=cov_class_2, allow_singular=True)
print(dist_class_1)

prior_class_1 =
len(class_1_data)/(len(class_1_data)+len(class_2_dat
a))

prior_class_2 =
len(class_2_data)/(len(class_1_data)+len(class_2_dat
a))

import numpy as np
from scipy.stats import multivariate_normal
from sklearn.metrics import accuracy_score

predicted_probabilities_class_1 =
dist_class_1.pdf(X_test)
predicted_probabilities_class_2 =
dist_class_2.pdf(X_test)

posterior_class_1 = (predicted_probabilities_class_1
* prior_class_1) / (predicted_probabilities_class_1 *
prior_class_1 + predicted_probabilities_class_2 *
prior_class_2)
posterior_class_2 = 1 - posterior_class_1

predictions =
np.argmax(np.vstack((predicted_probabilities_class_
1, predicted_probabilities_class_2)), axis=0)

accuracy = accuracy_score(y_test, predictions)

import numpy as np
from scipy.spatial.distance import euclidean
over_sampler = SMOTE(sampling_strategy='auto')
under_sampler =
RandomUnderSampler(sampling_strategy='auto')

class KNNClassifier:
    def __init__(self, k=5):
        self.k = k
        self.X_train = None
        self.y_train = None

    def fit(self, X_train, y_train):
        self.X_train = X_train
        self.y_train = y_train

    def predict(self, X_test):
        predictions = []

        for test_point in X_test:
            distances = [np.linalg.norm(test_point -
train_point) for train_point in self.X_train]
            neighbors_indices =
np.argsort(distances)[:self.k]
```

```
            class_counts =
np.bincount(self.y_train[neighbors_indices],
minlength=np.max(self.y_train) + 1)

            predicted_class = np.argmax(class_counts)

            predictions.append(predicted_class)
        report = classification_report(y_test,
np.array(predictions))
        print(k,accuracy_score(y_test,
np.array(predictions)))
        error_rate = 1 - accuracy

        return np.array(predictions)

accuracies = []
knn_detector = KNNClassifier(k=4)
knn_detector.fit(X_train.values,y_train.values)

anomaly_predictions =
knn_detector.predict(X_test.values)

import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.ensemble import IsolationForest
from sklearn.compose import ColumnTransformer
from sklearn.pipeline import Pipeline
from sklearn.metrics import accuracy_score

df = application_data_df

X = df.drop('TARGET', axis=1)
y = df['TARGET']

numeric_features = X.select_dtypes(include=['int',
'float']).columns
categorical_features =
X.select_dtypes(include=['object']).columns

X_train, X_test, y_train, y_test = train_test_split(X,
y, test_size=0.2, random_state=42)

numeric_transformer = StandardScaler()
categorical_transformer = 'passthrough'

preprocessor = ColumnTransformer(
    transformers=[
        ('num', numeric_transformer, numeric_features),
        ('cat', categorical_transformer,
categorical_features)
    ])

model = IsolationForest(contamination=0.05)

pipeline = Pipeline([
    ('preprocessor', preprocessor),
    ('model', model)
])

pipeline.fit(X_train)

predictions = pipeline.predict(X_test)

predictions_mapped = [1 if pred == -1 else 0 for pred
in predictions]

accuracy = accuracy_score(y_test,
predictions_mapped)

print(f"Accuracy: {accuracy:.2%}")
```